

ML-Based Decision Support System for Smart Investments

Dr. Reshma Koli¹, Avanish Vadke², Atharva Shelke³,
Abhishek Thormothe⁴, Aadit Suryawanshi⁵,
Prof. Harshada Sonkamble⁶

¹²³⁴⁵⁶A.P. Shah Institute of Technology, Thane, India
rrkoli@apsit.edu.in, avanishvadke3@apsit.edu.in,
atharvashelke2@apsit.edu.in, abhishekthormothe150@apsit.edu.in,
aadsuryavanshi60@apsit.edu.in, hksonkamble@apsit.edu.in

Abstract—Retail investors and discretionary traders make decisions under uncertainty, limited information, and behavioral bias; without a measurable workflow, these decisions can lead to avoidable drawdowns. This paper presents *QuantFin*, a machine learning-based decision support system for equity investments targeting Indian large-cap stocks (Nifty 50 constituents available in our dataset). *QuantFin* integrates: (i) reproducible ingestion of OHLCV time series from multi-header CSV exports; (ii) a technical-indicator feature pipeline; (iii) a multi-model prediction stack (ridge regression, logistic regression, support vector machines (SVM), ARIMA, and long short-term memory (LSTM)) used for forecasting and comparison; (iv) portfolio construction via ML-driven ranking and optimization-based baselines; and (v) walk-forward backtesting that evaluates rebalanced strategies against a benchmark under leakage-aware date cutoffs.

The system is implemented as a FastAPI backend exposing portfolio, analytics, prediction, and backtest endpoints, coupled with a React/TypeScript dashboard. We report results produced by the repository’s own training logs and backtest engine. For example, on the RELIANCE dataset, saved training metrics show test accuracy of 0.5003 (logistic regression) and 0.5136 (SVM) for directional classification, and LSTM test MAPE of 20.447% for price forecasting. In a reproducible walk-forward run (monthly rebalancing, 2023-01-01 to 2024-01-01), the implemented LINEAR strategy achieved CAGR 48.31% versus 30.60% for the benchmark under the repository’s assumptions.

Index Terms—Decision Support Systems, Quantitative Finance, Portfolio Construction, Walk-Forward Backtesting, Time Series Forecasting, FastAPI, Nifty 50

I. INTRODUCTION

Quantitative investing systems increasingly combine statistical modeling, machine learning, and portfolio construction under realistic evaluation. Recent IEEE and Springer literature covers both end-to-end portfolio decision support systems and algorithmic techniques for portfolio construction and ranking [1]–[3]. A recurring theme is that reported performance is sensitive to evaluation protocol: time-order must be respected and realistic constraints (e.g., rebalancing cadence and capital limits) must be explicit.

QuantFin is designed as a full-stack platform that connects these components for the Indian equity universe (Nifty 50 constituents available in the repository dataset). Unlike isolated notebooks, *QuantFin* operationalizes a repeatable workflow: (i) ingestion of OHLCV data from CSV exports with ro-

bust header handling; (ii) technical indicator generation; (iii) model training and prediction orchestration across multiple approaches (ridge regression, logistic regression, SVM, ARIMA, LSTM); (iv) portfolio construction and rebalancing; and (v) walk-forward backtesting with a benchmark.

Contributions. The main contributions of this work are:

- An end-to-end, modular architecture (FastAPI backend + React/Vite dashboard) for interactive quantitative analysis.
- A walk-forward backtesting procedure that enforces date cutoffs at each rebalance date (mitigating look-ahead bias) and reports risk-adjusted metrics.
- A multi-model prediction layer and multiple portfolio construction baselines (ML-driven ranking, model-weighted allocation, mean-variance, risk parity) implemented in the repository.

Paper organization. Section II describes system architecture; Section IV covers data and features; Section V presents the model suite; Sections VI–VII describe portfolio construction and backtesting; Section VIII reports experiments; Section IX concludes.

II. SYSTEM OVERVIEW

QuantFin is implemented as a modular full-stack system:

- **Backend:** a FastAPI service exposing endpoints for portfolio rebalancing, analytics (training + metrics), predictions, and walk-forward backtesting.
- **Model services:** reusable components for data preprocessing, feature engineering, model training, prediction orchestration, and portfolio allocation.
- **Frontend:** a React + TypeScript dashboard built with Vite that calls backend APIs and visualizes model performance, allocations, and time-series charts.

Figure 1 provides a conceptual architecture diagram.

III. API AND SERVICE DESIGN

QuantFin exposes modular API surfaces aligned to internal services.

- **Portfolio endpoints** (prefix `/api/portfolio`): summary and performance series, portfolio health, rebalancing workflows, and cached strategy comparisons.

QuantFin Architecture (conceptual).

Data layer: CSV OHLCV $\rightarrow$ preprocessing $\rightarrow$ feature engineering.
Model layer: Linear, Logistic, SVM, ARIMA, and LSTM models with an ensemble option.
Decision layer: portfolio weighting + rebalancing + back-testing.
Serving layer: FastAPI endpoints consumed by a React dashboard.

Fig. 1. QuantFin system architecture.

- **Analytics endpoints** (prefix `/api/analytics`): background model training triggers (with status reporting), retrieval of cached model metrics, and candlestick/price-series endpoints backed by local OHLCV.
- **Prediction endpoints** (prefix `/api/predictions`): per-symbol inference and portfolio recommendations parameterized by model family and prediction horizon.
- **Backtest endpoints** (prefix `/api/backtest`): walk-forward evaluation via POST /run, returning portfolio and benchmark value series plus risk/return metrics.

The API layer is intentionally thin; request/response validation is performed via typed request/response models.

IV. DATA AND FEATURE ENGINEERING

QuantFin operates on daily OHLCV time series for Nifty 50 constituents stored as CSV files (one file per symbol). In the current repository state, the backend data directory contains 49 stock CSV files (excluding auxiliary files such as `nifty50.csv` and `tcs_combined_news.csv`). While a news CSV exists, it is explicitly excluded from the backend's stock-data loader and is not used by the current prediction or allocation flow.

A. Preprocessing

Market CSV files can contain multi-row headers depending on the export source. QuantFin includes preprocessing routines that normalize these files into a canonical schema with columns (Date, Open, High, Low, Close, Volume) and a datetime index. For example, the real-data service reads each symbol CSV with header skipping (`skiprows=3`), coerces numeric columns, drops invalid rows, sorts by date, and caches the result to reduce repeated I/O.

B. Feature set

The feature pipeline generates both return-based and indicator-based features commonly used in technical analysis: simple and exponential moving averages (SMA/EMA), MACD, RSI, Bollinger bands, rolling volatility, lagged returns, momentum features, and volume features. In addition, log-returns are computed for stability in statistical modeling and walk-forward evaluation pipelines.

Given close price P_t , QuantFin uses simple returns and log-returns:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}, \quad \ell_t = \log \left(\frac{P_t}{P_{t-1}} \right) \quad (1)$$

These features are combined with indicators to form the model input matrix.

V. MODEL SUITE AND PREDICTION ORCHESTRATION

QuantFin supports complementary model families to address both regression and classification tasks.

A. Ridge regression (linear)

For regression targets, QuantFin trains a linear model with ℓ_2 regularization (ridge regression). Given features $X \in \mathbb{R}^{n \times d}$ and targets $y \in \mathbb{R}^n$, ridge solves:

$$\hat{\beta} = \arg \min_{\beta} \|y - X\beta\|_2^2 + \alpha \|\beta\|_2^2 \quad (2)$$

where $\alpha > 0$ controls shrinkage. This choice is computationally efficient and mitigates multicollinearity among correlated indicators. where $\alpha > 0$ controls shrinkage. This choice is computationally efficient and mitigates multicollinearity among correlated indicators.

B. Logistic regression (directional classification)

For directional classification (up/down), QuantFin trains logistic regression. With feature vector x and label $y \in \{0, 1\}$, the model estimates:

$$\Pr(y = 1 | x) = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}} \quad (3)$$

and optimizes regularized cross-entropy. This provides an interpretable classification baseline.

C. Support vector machines

Support vector machines (SVMs) provide a non-linear classification baseline via kernel methods. QuantFin includes an RBF-kernel SVM pipeline trained on engineered technical features. In the saved RELIANCE training log, the best hyperparameters were $C = 10.0$ and $\gamma = 0.01$.

D. ARIMA

ARIMA family models provide a classical statistical baseline for univariate time series. QuantFin includes an ARIMA training routine as a comparative baseline, consistent with recent studies that benchmark ARIMA alongside neural approaches [4].

E. LSTM

LSTM sequence models capture non-linear temporal patterns and can ingest windows of engineered features. QuantFin provides an LSTM implementation including sequence windowing, scaling, and early stopping. Recent IEEE work

has applied LSTM-based architectures to stock trend prediction under time-ordered market data [5]. For a sequence $(x_{t-L+1}, \dots, x_t)$, an LSTM cell computes gated updates:

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f), \quad i_t = \sigma(W_i[x_t, h_{t-1}] + b_i), \quad (4)$$

$$\tilde{c}_t = \tanh(W_c[x_t, h_{t-1}] + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (5)$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o), \quad h_t = o_t \odot \tanh(c_t) \quad (6)$$

where f_t, i_t, o_t are forget/input/output gates and c_t is the cell state.

F. Prediction service

A central prediction service loads trained models, generates per-symbol forecasts for a chosen horizon, and aggregates predictions for downstream allocation. The API supports both single-model inference and an “ensemble” option that aggregates signals across available model families. Ranking-based modeling and selection are commonly used in stock recommendation pipelines [3].

VI. PORTFOLIO CONSTRUCTION AND REBALANCING

QuantFin exposes portfolio construction via both ML-driven ranking and optimization-based weighting.

A. Model-driven ranking

The repository implements ML-driven ranking in the portfolio allocator used by the `/api/portfolio/rebalance` endpoint. For each symbol, the allocator estimates a predicted return (in percent) and a confidence score derived from data characteristics (volatility regime, trend consistency, and data completeness). It ranks symbols by:

$$\text{score}_i = \hat{r}_i \cdot c_i \quad (7)$$

filters out negative predicted returns, selects the top- N symbols, and converts weights into integer share quantities under a capital budget.

B. Mean-variance optimization

For users who prefer optimization-based weighting, QuantFin supports mean-variance and risk-parity style allocations as baselines. In the current repository state, the prediction service’s optimization routines use a simplified covariance structure (per-asset volatility estimates with a fixed pairwise correlation assumption), while the walk-forward backtest engine can estimate a covariance matrix empirically from historical returns over a rolling lookback window. Mean-variance style formulations remain a widely used baseline family in portfolio optimization research [2].

C. Rebalance procedure

Rebalancing produces a set of target weights and quantities for a specified capital allocation. The backend persists portfolio metadata and exposes endpoints for summary and performance.

Algorithm 1 Leakage-aware monthly rebalancing (simplified)

Input: symbols S , rebalance dates D , horizon h , top- N .
Initialize capital C_0 .
for $t = 1$ to $|D| - 1$ **do**
 Cutoff date $d \leftarrow D[t]$.
 For each $s \in S$: compute features using data $\leq d$ and predict return $\hat{r}_s$.
 Compute confidence c_s from data quality/volatility/trend fit.
 Rank by score $\hat{r}_s \cdot c_s$ and select top- N with $\hat{r}_s > 0$.
 Compute target weights and integer share quantities under capital C_{t-1} .
 Realize return over $(D[t], D[t+1])$ and update capital.
end for
Output: portfolio value series and returns.

VII. WALK-FORWARD BACKTESTING

Backtesting is implemented as a walk-forward simulation: at each rebalance date, the system generates predictions using only historical data up to that date, constructs a portfolio, and then evaluates realized returns over the next period. This avoids look-ahead bias by construction.

QuantFin reports standard risk-adjusted metrics such as CAGR, Sharpe ratio, Sortino ratio, maximum drawdown, annualized volatility, Calmar ratio, and win rate. A benchmark portfolio is computed using an equal-weight basket over the available universe for the same rebalance periods. Periodic rebalancing and performance evaluation are standard in recent ML-based portfolio management studies [1], [6].

Given a portfolio value series V_t and periodic returns $r_t = \frac{V_t - V_{t-1}}{V_{t-1}}$, the implemented evaluation reports:

$$\text{CAGR} = \left(\frac{V_T}{V_0} \right)^{\frac{1}{Y}} - 1, \quad \sigma = \sqrt{\text{Var}(r_t)} \cdot \sqrt{K} \quad (8)$$

where Y is the number of years in the backtest and K is the annualization factor for the return frequency. Sharpe and Sortino use excess returns over a risk-free rate r_f :

$$\text{Sharpe} = \frac{\mathbb{E}[r_t - r_f/K]}{\sigma}, \quad \text{Sortino} = \frac{\mathbb{E}[r_t - r_f/K]}{\sigma_-} \quad (9)$$

where σ_- is downside deviation computed over $\min(r_t, 0)$.

VIII. EXPERIMENTAL RESULTS

This section summarizes representative training results generated by the QuantFin training pipeline.

A. Training configuration

Models are trained on engineered technical features and evaluated on held-out data. For time-series workflows, a time-based split is preferred to avoid leakage through shuffling.

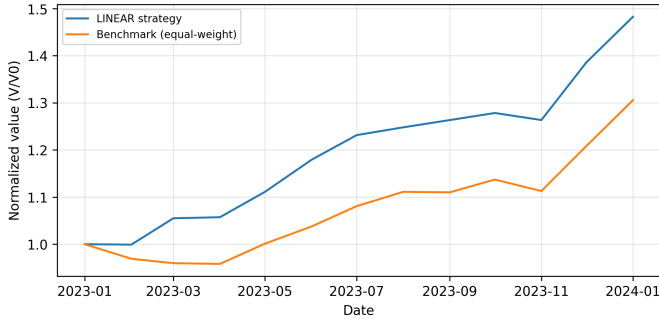


Fig. 2. Normalized equity curve for the implemented LINEAR strategy vs. the equal-weight benchmark (monthly rebalancing, 2023-01-01 to 2024-01-01).

B. Reproducible artifacts (repo-grounded)

QuantFin persists intermediate artifacts that make experimentation reproducible within the repository: (i) cached training summaries and per-symbol metrics in JSON files (e.g., `backend/training_results.json`, `backend/training_summary.json`); (ii) validation outputs (`backend/validation_report.json`); and (iii) saved model weights for some families (e.g., pre-trained LSTM .h5 files under `backend/models`).

C. Sample model metrics

Table I reports sample metrics produced by QuantFin for the RELIANCE symbol (from an automated training run recorded in the repository). While these baselines are not optimized for alpha generation, they validate the end-to-end training and evaluation pipeline.

D. Walk-forward backtest example

We also ran the repository’s walk-forward backtest engine with monthly rebalancing from 2023-01-01 to 2024-01-01 using the implemented LINEAR strategy (top 10, initial capital 100000). Under the repository’s assumptions (no transaction costs or slippage), the strategy achieved CAGR 48.31% versus 30.60% for the benchmark, Sharpe 3.04 versus 1.75, and maximum drawdown -1.17% versus -4.18%. Final portfolio value was 148,271 versus 130,578 for the benchmark.

Figure 2 visualizes the realized equity curve over the rebalance dates, and Figure 3 shows the corresponding drawdown profiles for the strategy and benchmark.

E. Discussion

The reported metrics highlight known challenges in financial prediction: noisy labels, regime shifts, and weak signal-to-noise ratios. Negative test R^2 for the regression baseline indicates that naive baselines can outperform poorly specified targets on some splits. QuantFin mitigates common failure modes by making preprocessing and evaluation explicit and repeatable, exposing multiple model families for comparison, and emphasizing risk-adjusted backtest metrics rather than a single accuracy value.

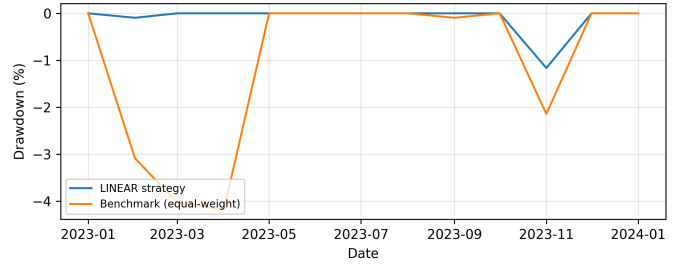


Fig. 3. Drawdown comparison for the implemented LINEAR strategy vs. the equal-weight benchmark over the same walk-forward run.

F. Limitations observed in the current codebase

QuantFin is intended as an end-to-end educational and prototyping platform, and the repository reflects this stage. For example, the backend validation report included in the repository indicates some integration checks are not passing, and production readiness is explicitly marked as false. In addition, backtests currently omit explicit transaction cost and slippage modeling, which can materially affect realized performance.

G. Threats to validity

While QuantFin enforces time-ordered cutoffs in walk-forward evaluation, several threats remain that can inflate or destabilize results if not addressed: (i) the absence of transaction costs, slippage, and liquidity constraints in the current backtest; (ii) simplified covariance assumptions in some optimization baselines (e.g., constant-correlation covariance in the prediction service); and (iii) model-selection and hyperparameter choices that may implicitly tune to a particular market regime. These limitations are documented here to keep interpretation aligned with the repository’s current implementation.

TABLE I
SAMPLE TRAINING METRICS FOR A REPRESENTATIVE SYMBOL (RELIANCE) FROM A RECORDED QUANTFIN RUN.

Model	Test Acc.	Error (MAE/MAPE)	Notes
Logistic Regression	0.5003	–	Test AUC: 0.5073; test F1: 0.2779
SVM (RBF)	0.5136	–	Best params: $C = 10.0, \gamma = 0.01$; test AUC: 0.5051
Ridge Regression	–	MAE: 0.0742	$\alpha = 100.0$; test $R^2 = -0.3354$
LSTM	–	MAPE: 20.447%	Seq len: 60; units: [128,64]; dropout: 0.2

TABLE II
WALK-FORWARD BACKTEST SUMMARY (MONTHLY REBALANCING,
2023-01-01 TO 2024-01-01).

Portfolio	CAGR	Sharpe	Max DD
LINEAR Strategy	48.31%	3.04	-1.17%
Benchmark (equal-weight)	30.60%	1.75	-4.18%

IX. CONCLUSION

This paper presented QuantFin, a machine learning–based decision support system for investments that integrates data preprocessing, feature engineering, multi-model prediction, portfolio construction, and walk-forward backtesting for Nifty 50 equities available in the repository dataset. The system provides practical APIs and an interactive dashboard, enabling users to compare models, rebalance portfolios, and validate performance against a benchmark.

Current results demonstrate that the end-to-end pipeline is functional and produces measurable baselines. The platform’s outputs should be interpreted as decision support rather than a guarantee of profit: predictive performance remains limited by market non-stationarity, and the backtest currently omits transaction costs and slippage. Future work includes improving label design, adding transaction-cost modeling, and integrating the repository’s auxiliary news dataset into the feature pipeline only once it is wired into the active prediction/portfolio flow.

REFERENCES

- [1] P. K. Aithal, M. Geetha, D. U. B. Savitha, and P. Menon, “Real-time portfolio management system utilizing machine learning techniques,” *IEEE Access*, vol. 11, pp. 32 595–32 608, 2023.
- [2] A. Gunjan and S. Bhattacharyya, “A brief review of portfolio optimization techniques,” *Artificial Intelligence Review*, vol. 56, no. 5, pp. 3847–3886, 2022.
- [3] S. Saha, J. Gao, and R. Gerlach, “Stock ranking prediction using list-wise approach and node embedding technique,” *IEEE Access*, vol. 9, pp. 88 981–88 996, 2021.
- [4] Z. Gao, “Stock price prediction with arima and deep learning models,” in *2021 IEEE 6th International Conference on Big Data Analytics (ICBDA)*, 2021, pp. 61–68.
- [5] V. Karlis, K. Lepenioti, A. Bousdekis, and G. Mentzas, “Stock trend prediction by fusing prices and indices with lstm neural networks,” in *2021 12th International Conference on Information, Intelligence, Systems & Applications (IISA)*, 2021, pp. 1–7.
- [6] Q. Y. E. Lim, Q. Cao, and C. Quek, “Dynamic portfolio rebalancing through reinforcement learning,” *Neural Computing and Applications*, vol. 34, no. 9, pp. 7125–7139, 2022.